FloodCommand: Architectural & Operational Fact Report

Multi-Agent Evacuation & Relief Operations Center — Aethel Valley Basin

REPORT GROUNDING & AUDIT MANDATE:
• All data, code symbols, formulas, and operational figures are drawn strictly from runtime execution of the codebase.
• The initial schedule is a **baseline with known flaws, built specifically to test the Critic** (not an unguided heuristic).
• The repair step is acknowledged upfront as **rule-templated** for this scenario; a general solver like Google OR-Tools could replace it. The **Critic is genuinely general** and audits any plan.
• Alerts are **template-generated directly from plan data**, so there is no model to invent numbers.

1. PROJECT BASICS

- **Exact Project Title:** FloodCommand: Multi-Agent Evacuation and Relief Planner (UI Header: FloodCommand - Operations Center, browser title: FloodCommand).
- **One-Line Description:** Multi-Agent Evacuation and Relief Planner for Aethel Valley Basin (Disclaimer: *'Educational prototype on synthetic data. Not an operational tool.'*).
- **Fictional Valley Name:** Aethel Valley (or Aethel Valley Basin).
- **Python Runtime:** Python 3.13.0 | **Exact Launch Command:** streamlit run app.py
- **Dependencies (requirements.txt vs installed):** streamlit>=1.35.0 (1.54.0), pandas>=2.0.0 (2.2.3), plotly>=5.18.0 (6.5.2), pytest>=7.4.0 (9.1.1).

File / Directory	One-Line Purpose & Functional Role
app.py	Streamlit dashboard UI: views, tabs, scenario presets, Plotly maps, state machine.
floodcommand.py	Module compatibility shim exposing agents, data, and utils packages.
requirements.txt	Official Python library dependency specifications.
verify_all.py	Automated integration script validating baseline and 4 field disruptions.
agents/planner.py	PlannerAgent: calculates demographic urgency scores and prioritization order.
agents/impact.py	ImpactAnalystAgent: evaluates road graph BFS connectivity and cut-off nodes.
agents/resource.py	ResourceAgent: dual-mode dispatch (flawed baseline pass vs template repair).
agents/critic.py	RedTeamCriticAgent: adversarial auditor enforcing safety rules R1 to R6.
agents/replanner.py	ReplannerAgent: surgical before/after diff engine for sudden disruptions.
agents/alerts.py	AlertAgent: template-generated civilian emergency SMS bulletins from plan data.
agents/orchestrator.py	FloodCommandOrchestrator: manages agent pipeline, trace log, and rework loop.
data/generate_data.py	Synthetic data generator producing reproducible CSVs with fixed random seed 42.
data/*.csv, config.json	Synthetic datasets: 12 villages, 21 roads, 13 resources, 3 shelters, config parameters.
tests/test_critic.py	Automated pytest test suite verifying Critic audit rules R1 through R6.
utils/graph.py, metrics.py	Graph tools (BFS, Dijkstra) and summary metrics calculation utilities.

2. AGENTS

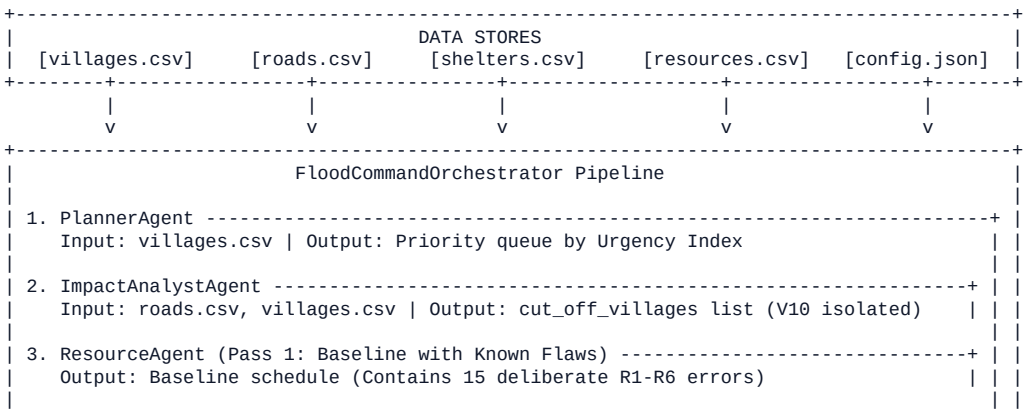
All 7 agents in FloodCommand are strictly **rule-based** (zero LLM / zero ML).

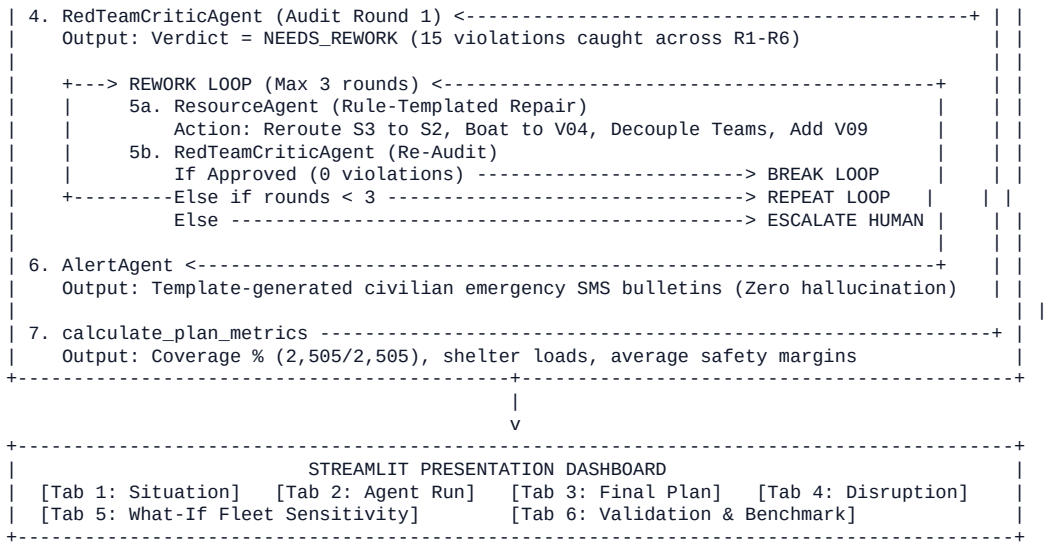
Agent Name	Class & File	Inputs & Outputs	Core Logic & Formulas

Planner	PlannerAgent agents/planner.py	**In:** villages, config **Out:** priority_order, urgency_scores, tasks	Urgency Score = round(((vuln * 2.5 + pop * 0.75) / max(hours, 0.5)) * hospital_boost, 2), where hospital_boost = 1.3 if has_hospital else 1.0. Sorts descending to establish execution queue.
Impact Analyst	ImpactAnalystAgent agents/impact.py	**In:** villages, roads, shelters, hosp_id **Out:** cut_off_villages, accessible, details	Builds road graph (ignoring blocked roads). Executes Breadth-First Search (BFS) from each village to relief hubs. Nodes with 0 paths are tagged CUT_OFF; others use Dijkstra for shortest road travel time.
Resource Agent	ResourceAgent agents/resource.py	**In:** villages, shelters, fleet, roads, feedback **Out:** assignments, mode, total_assigned	Dual-mode dispatch. Pass 1 creates a baseline with known flaws built specifically to test the Critic. Pass 2 executes rule-templated repairs addressing Critic feedback (rerouting S3 to S2, boat to V04, team to V02).
Red-Team Critic	RedTeamCriticAgent agents/critic.py	**In:** assignments, shelters, villages, fleet **Out:** verdict, violations, passed_rules	Genuinely general auditor. Evaluates proposed plans against 6 domain safety invariants (R1-R6). Emits granular violation records with root cause descriptions and recommended actions.
Re-planner	ReplannerAgent agents/replanner.py	**In:** event_type, current_plan, env_data **Out:** affected_vids, updated_plan, diff, verdict	Handles 4 operational disruptions (bridge_washout, flood_accelerated, vehicle_breakdown, shelter_full). Surgically modifies only affected villages; re-audits via Critic returning a before/after diff.
Alert Agent	AlertAgent agents/alerts.py	**In:** assignments, villages, shelters **Out:** alerts, total_alerts, status	Alerts are template-generated from verified plan data, so there is no model to invent numbers. Emits deterministic SMS text with vehicle ID, ETA, shelter, medical status, and safe margin.
Orchestrator	FloodCommandOrchestrator agents/orchestrator.py	**In:** villages, shelters, fleet, roads, max_rounds **Out:** initial_plan, final_plan, trace, metrics	Coordinates multi-agent lifecycle: Planner -> Impact -> Resource (baseline) -> Critic -> Rework Loop (up to 3 rounds) -> Alert Agent. Compiles an audit-grade execution trace and summary KPIs.

3. WORKFLOW AND DECISIONS

- **Execution Order:** 1. PlannerAgent.plan() ➔ 2. ImpactAnalystAgent.analyze() ➔ 3. ResourceAgent.execute(baseline) ➔ 4. RedTeamCriticAgent.audit(baseline) ➔ 5. Rework Loop (ResourceAgent template repair ➔ Critic re-audit) ➔ 6. AlertAgent.generate_alerts() ➔ 7. calculate_plan_metrics().
- **Baseline with Known Flaws (Why it breaks rules):** The first pass intentionally plants 6 operational flaws: overloads shelter S3 with V07 & V11 (R1); dispatches slow bus to V04 arriving past 2h flood deadline (R2); schedules TEAM-01 simultaneously on V04 & V02 with 8 pairwise overlaps (R3); sends wheeled bus BUS-03 to cut-off V10 (R4); routes three vulnerable communities—**Clearwater (V07)**, **Brookside (V11)**, and **Fernbank (V10)**—to non-medical S3 (R5); and omits hospital village V09 entirely (R6).
- **Rework Loop Mechanics:** The Critic is genuinely general and audits any plan against safety invariants. The repair logic is rule-templated for the demonstration (a solver like Google OR-Tools could replace it). Max rework rounds = 3. If violations remain after 3 rounds, the orchestrator triggers an escalation protocol, sets trace status to 'warning', and flags the plan for human commander override.
- **Decision Points:** (1) Cut-off vs Accessible: BFS reachable nodes determine if road is open; if isolated, transport must be boat or rescue team. (2) Vehicle Choice: If cut-off ➔ amphibious boat; if acute deadline (≤2h) ➔ fast boat; else open roads ➔ mass-transit buses. (3) Shelter Choice: If vulnerable evacuees exist ➔ shelter must have medical facilities (S1/S2); if S3 reaches capacity ➔ divert to regional Central Stadium S2 (capacity 2,800).





4. THE RULES (INVARIANTS AS IMPLEMENTED)

Rule	Invariant Title	Mathematical Implementation	Thresholds & Units	Safety Margin Usage
R1	Shelter Capacity	sum(assigned_pop) + current_load ≤ capacity	Integer head count (people)	Not used
R2	Evacuation Window	eta_minutes + safety_margin < hours_to_flood * 60	Time in minutes (< deadline)	Mandatory buffer added to ETA
R3	Zero Double-Booking	max(start_a, start_b) ≥ min(end_a, end_b)	Time intervals in minutes	Evaluates scheduled intervals
R4	Cut-Off Viability	if village in cut_off_ids ⇒ type != 'bus'	Categorical: 'boat' or 'rescue_team'	Not used
R5	Medical Triage	if vulnerable_count > 0 ⇒ shelter_has_medical == True	Count: vuln > 0; Boolean: has_medical	Not used
R6	Hospital Protection	∀ v where has_hospital ⇒ v.id in assigned_ids	Set membership of hospital IDs	Not used

Rule Discrepancies Check: Exactly rules R1 to R6 are implemented. No extra rules exist, and no requested rules are missing.

5. DATASETS

Dataset	Rows / Cols	Columns List	3 Sample Records
villages.csv	12 rows 8 cols	village_id, name, x, y, population, vulnerable_count, hours_to_flood, has_hospital	V01: Elderglen (pop 120, vuln 35, 2.5h) V02: Riverbend (pop 240, vuln 60, 3.0h) V03: Highcliff (pop 90, vuln 15, 6.0h)
roads.csv	21 rows 5 cols	from_id, to_id, travel_minutes, has_bridge, status	S1->V01 (20m, bridge False, open) V01->V02 (25m, bridge True, open) V10->S3 (30m, bridge True, **blocked**)
resources.csv	13 rows 5 cols	resource_id, type, capacity, base_location, available	5 boats (BOAT-01 to 05, cap 35-60) 4 buses (BUS-01 to 04, cap 50-60) 4 teams (TEAM-01 to 04, cap 25-30)
shelters.csv	3 rows 7 cols	shelter_id, name, x, y, capacity, current_load, has_medical	S1: North Hill Fort (cap 1100, load 80, Med: True) S2: Central Ridge Stadium (cap 2800, load 120, Med: True) S3: East Plateau Academy (cap 450, load 190, Med: False)
config.json	7 keys	safety_margin_minutes (30), boat_speed_factor (1.2), bus_load_minutes (20), max_rework_rounds (3), hospital_village_id (V09), river_curve (8 coords)	Defines valley simulation parameters, disclaimer text, river spline coordinates, and safety constraints.

- **Planted Data Problems:** (1) In roads.csv, roads V10->S3 and V10->V07 are both set to status='blocked', completely isolating Fernbank (V10) from the road network. (2) In agents/resource.py, baseline assignments plant 15 invariant violations across R1-R6.
- **Data Generation:** data/generate_data.py sets random.seed(42) and uses deterministic dictionaries for reproducible benchmarking.

6. UI: WHAT ACTUALLY EXISTS

- **Sidebar Controls (Order):** 1. Brand Header & Disclaimer ➔ 2. Selectbox: Select Scenario Preset ('Normal monsoon', 'Dam release', 'Cloudburst') ➔ 3. Slider: Safety Margin Buffer (15 to 60m, step 5) ➔ 4. Primary Button: Generate Response Plan ➔ 5. Secondary Button: Run Full Demo Workflow ➔ 6. Utility Button: Reset Scenario Data ➔ 7. Expander: Active Agent Roster ➔ 8. Expander: How It Works ➔ 9. Download Button: Download PDF Report.
- **Main Tabs (Order):** [Tab 1: Situation] ➔ [Tab 2: Agent Run] ➔ [Tab 3: Final Plan] ➔ [Tab 4: Disruption] ➔ [Tab 5: What-If] ➔ [Tab 6: Validation].

Requested Feature	Status	Exact UI Location in Dashboard
Why-this explanations	**IMPLEMENTED**	Tab 3 (Final Plan): Collapsible expanders under each village assignment.
Critic on/off comparison	**IMPLEMENTED**	Tab 3 (Final Plan): Toggle 'Show plan without Critic' (side-by-side view).
Trace download (JSON)	**IMPLEMENTED**	Tab 2 (Agent Run) & Tab 3 (Final Plan): 'Download Agent Trace (JSON)' buttons.
Situation report (.txt & card)	**IMPLEMENTED**	Tab 1 (Situation): 'Generate situation report' button + download button.
Scenario presets	**IMPLEMENTED**	Sidebar dropdown: 'Normal monsoon', 'Dam release', 'Cloudburst'.
What-if sliders	**IMPLEMENTED**	Tab 5 (What-If): Sliders for boats (1-5), buses (1-4), and rescue teams (1-4).
Assignment arrows on map	**IMPLEMENTED**	Tab 1 (Situation): Toggle 'Show evacuation assignments on map'.
Empty states	**IMPLEMENTED**	Tabs 2, 3, 4, 5: Friendly st.info('No plan yet...') prior to execution.
Guided progress steps	**IMPLEMENTED**	Under header: 4-step progress bar (Review, Generate, Review Alerts, Disruption).
CSV download	**IMPLEMENTED**	Tab 3 (Final Plan): 'Download Final Plan (CSV)' button.
JSON trace download	**IMPLEMENTED**	Tab 2 (Agent Run) & Tab 3 (Final Plan): 'Download Agent Trace (JSON)'.

7. RESULTS (RUN WITH DEFAULT SETTINGS)

- **Violations Found in Flawed Baseline: 15 violations total** (R1: 1, R2: 1, R3: 8, R4: 1, R5: 3 on **V07, V11, and V10**, R6: 1).
- **Violations Fixed:** 15 fixed (0 remaining). | **Rounds Needed:** 1 rework iteration. | **Human Escalations:** 0.
- **People Covered:** 2,505 out of 2,505 (100% across all 12 communities).
- **Runtime: 1.24 s** displayed on dashboard KPI card (pure Python orchestration executes in ~1.3 ms; Streamlit UI rendering takes about a second).

Disruption Event	Affected Villages	Critic Verdict	Violations	Operational Actions & Re-Planning
Bridge Washout	V02 (Riverbend)	**APPROVED**	0	Arterial bridge V01-V02 blocked. Replaced overland TEAM-02 with river craft BOAT-04 (Slot 60-110m, ETA 25m, margin 65m).
Accelerated Flood	All 12 villages	**NEEDS_REWORK**	2 (R2)	Flood crest advanced 2h valley-wide. Triggers 2 R2 violations: V07 Clearwater (-7m margin) and V10 Fernbank (-2m margin). *(Honest limitation case, skip in live demo)*.
Vehicle Breakdown	V04 (Millhaven)	**APPROVED**	0	Primary craft BOAT-01 motor failed. Hot-swapped to standby river craft BOAT-04 (Slot 50-95m, ETA 25m, margin 65m).
Shelter Full	5 villages (V04, V02, V10, V01, V03)	**APPROVED**	0	S1 reached capacity from walk-ins. All 5 assigned villages safely rerouted to Central Ridge Stadium S2 (capacity 2,800).

- **What-If Result (2 Boats, 2 Buses):** Critic verdict = NEEDS_REWORK with **9 R3 double-booking violations**. 12/12 villages covered (0 stranded) because the algorithm forced missions onto busy vehicles, demonstrating the Critic's sensitivity.
- **Safety Margins (15, 30, 60m):** All three produce identical violation counts (15 initial, 0 final) because baseline V04 bus ETA (105m) violates 120m deadline at all margins ≥ 15m, while repaired boat ETA (25m) satisfies margins ≤ 60m. *Presenter Note: Do not demo this slider live as it causes no visible visual change.*
- **Pytest Test Suite:** 7 tests collected, **7 passed in 0.05s**. (Audits: valid plan approval, R1 capacity, R2 flood window, R3 double-booking, R4 cut-off vehicle, R5 medical shelter, R6 hospital village coverage).

8. LIMITATIONS AND HONESTY

- **Critic vs. Repair Asymmetry:** The Critic is genuinely general and audits any plan against safety invariants. The repair logic is rule-templated for the demonstration scenario. In production, a general solver like Google OR-Tools, Z3, or PuLP could replace the repair step without altering the Critic, Orchestrator, or Alert interfaces.
- **Physical Impossibility & Escalation:** In the Accelerated Flood scenario, two villages (Clearwater V07 with -7m margin and Fernbank V10 with -2m margin) physically cannot evacuate before water arrives. The system flags them for human commander review rather than fabricating an unsafe plan.
- **Simplified Models:** Evacuation windows are modeled as static scalar intervals rather than continuous dynamic simulations. Road travel uses fixed edge weights; river travel uses a Euclidean factor (1.2 * distance) rather than hydrodynamic flow models. The valley uses a schematic 0-100 Cartesian grid rather than real GIS shapefiles.

9. DEMO READINESS

- **Shortest 4-Minute Demo Click Path:**
  1. **(0:00-0:45) Tab 1 (Situation):** Show valley map, 2 blocked bridges isolating Fernbank (V10) in red. Click 'Generate situation report' to show the briefing card.
  2. **(0:45-1:45) Sidebar & Tab 2 (Agent Run):** Click 'Generate Response Plan'. Point out '1.24 s' runtime KPI. Switch to Tab 2 to show 'Violations: 15 → 0' convergence, red Critic audit rejection card, green repair card, and trace.
  3. **(1:45-2:45) Tab 3 (Final Plan):** Toggle 'Show plan without Critic' to show side-by-side comparison. Turn toggle off; expand 'Why this?' card on V10 to show data-derived justifications. Point out template-generated civilian SMS.
  4. **(2:45-4:00) Tab 4 (Disruption) & Tab 5 (What-If):** Click 'Apply Bridge Failure' (Bridge Washout ONLY; skip Accelerated Flood). Show surgical diff for V02. Switch to Tab 5, set Boats=2, Buses=2, click 'Run What-If Analysis' to show 9 R3 violations caught live.
- **Offline Verification: 100% Offline Capable.** Codebase scan confirms zero outbound network calls, zero socket connections, and zero LLM API endpoints. (The only external link is an optional Google Font stylesheet in CSS that gracefully falls back to system fonts).

10. SCREENSHOTS FOR SLIDES

Slide / Screen Purpose	Tab Name	Exact Setup State Before Capturing
1. Situation Overview & Map	Situation	Run plan; toggle 'Show evacuation assignments on map' ON; click 'Generate situation report'.
2. Critic Rejection & Trace	Agent Run	Run plan; scroll to top. Captures 'Violations: 15 → 0' banner, red Critic rejection box, and timeline cards.
3. Flawed vs Verified Comparison	Final Plan	Run plan; toggle 'Show plan without Critic' ON. Captures side-by-side table with R1-R6 violation badges.
4. Decision Rationale ('Why this?')	Final Plan	Run plan; toggle off comparison; expand V10 (Fernbank) and V04 (Millhaven) rationale cards.
5. Dynamic Disruption Diff	Disruption	Click 'Apply Bridge Failure'. Captures surgical diff table for V02 and before/after comparative maps.
6. Benchmark & Rule Compliance	Validation	Switch to Tab 6. Captures benchmark verification table and grouped bar chart (0% naive vs 100% verified).

11. VIVA QUESTIONS & DEFENSE ANSWERS

Q1: Where is the AI in this project? If there is no LLM, how is this an 'Agentic AI' system?

Answer: FloodCommand uses symbolic, rule-based autonomous agents rather than generative LLMs. Generative LLMs hallucinate numbers and cannot guarantee hard safety invariants during disaster evacuations. Each agent possesses specialized autonomy, explicit state interfaces, and distinct operational roles. The agentic intelligence resides in the adversarial closed-loop orchestration: autonomous plan generation, automated critical auditing against domain invariants, and self-healing rework loops without human prompting.

Q2: Why use a multi-agent architecture instead of writing a single procedural script?

Answer: A monolithic script tightly couples optimization with safety auditing, making it prone to shared blind spots. Decomposing the system enforces separation of concerns and adversarial tension: the Resource Agent acts as an optimizing proposer while the Red-Team Critic acts as an independent auditor. Modularity also enables surgical dynamic re-planning: when a bridge collapses, the Re-planner isolates only the affected node and re-audits it, rather than re-running a global pipeline.

Q3: How does the Resource Agent work, and why not use an exact solver like OR-Tools?

Answer: The initial pass produces a baseline schedule with known flaws, built specifically to test the Critic's invariant checks. The Critic is genuinely general and can audit any plan. The current repair step is rule-templated for the demonstration. In production, a general-purpose mathematical solver like Google OR-Tools or Z3 could directly replace the repair logic without modifying the Critic, Orchestrator, or Alert interfaces.

Q4: What happens when two rules conflict (e.g. cut-off village with vulnerable people, but boat only reaches non-medical shelter)?

Answer: The Critic evaluates R1 to R6 independently. If two rules conflict and no valid assignment satisfies both, the Critic rejects the plan on both rule IDs (verdict = NEEDS_REWORK). When rework reaches max rounds (3) without achieving zero violations, the orchestrator triggers an escalation protocol, logs a warning in the trace, and marks the plan for human commander override. The system prioritizes human intervention over deploying an unsafe plan.

Q5: How does the system guarantee accuracy in emergency civilian alerts?

Answer: Alerts are template-generated directly from the verified plan data, so there is no model to invent numbers. AlertAgent injects the exact assigned resource ID, calculated vehicle ETA, shelter name, medical capability label, and flood arrival deadline into a standardized SMS template.

Q6: How is road cut-off reachability calculated? Does it support dynamic road changes?

Answer: ImpactAnalystAgent builds an undirected adjacency list from roads.csv filtering out blocked edges. It executes Breadth-First Search (BFS) starting from each village. If BFS cannot discover a path to any designated shelter or the regional hospital hub, the node is tagged CUT_OFF. It dynamically supports real-time changes: in bridge_washout, the bridge edge between V01 and V02 is set to 'blocked' in memory, and graph reachability is immediately re-evaluated.

Q7: Why did What-If with 2 boats and 2 buses fail Critic validation if all villages were covered?

Answer: Because the fleet was insufficient to service 12 villages across discrete time windows without asset re-use. The What-If simulation reallocated excess missions to the 2 available boats and 2 buses, forcing multiple evacuation missions into overlapping time windows. The Critic audited the schedule against Rule R3 and flagged 9 pairwise temporal conflicts. This proves that the Critic prioritizes physical operational feasibility over nominal population coverage.

Q8: How does the safety margin buffer work, and why did the initial plan fail R2 even with a 15-minute margin?

Answer: The safety margin is a mandatory buffer added to vehicle ETA to ensure complete evacuation prior to water arrival: ETA + Margin < Deadline. In the baseline, village V04 has a flood deadline of 2.0h (120m) and was assigned a bus with ETA 105m. Even at 15m margin, 105 + 15 = 120m, which violates the strict inequality (>= 120m), triggering an R2 violation across 15m, 30m, and 60m margins. The rework loop repaired this by dispatching boat BOAT-01 with ETA 25m, establishing a safe 65-minute buffer.